In [30]:
```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_rep
```

In [31]:
```
print(df.isnull().sum())
```

```
RespondentID                      0
Check_Weather_Daily               0
Check_Weather_Method              0
Weather_Smartwatch_Likelihood     0
Age                             919
Gender                            0
Household_Income                  0
US Region                         0
dtype: int64
```

In [32]:
```
print("Number of duplicate records:", df.duplicated().sum())
```

```
Number of duplicate records: 0
```

In [33]:
```
df = df.drop_duplicates()
```

In [34]:
```
print(df.columns.tolist())
```

```
['RespondentID', 'Check_Weather_Daily', 'Check_Weather_Method', 'Weather_Smartwat
ch_Likelihood', 'Age', 'Gender', 'Household_Income', 'US Region']
```

In [35]:
```
df.columns = df.columns.str.strip()

print(df.columns.tolist())
```

```
['RespondentID', 'Check_Weather_Daily', 'Check_Weather_Method', 'Weather_Smartwat
ch_Likelihood', 'Age', 'Gender', 'Household_Income', 'US Region']
```

In [36]:
```
weather_col = [col for col in df.columns if 'Weather' in col]

print(weather_col)
```

```
['Check_Weather_Daily', 'Check_Weather_Method', 'Weather_Smartwatch_Likelihood']
```

In [37]:
```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

df = pd.read_csv("weather-check.csv")

df.columns = df.columns.str.strip()

print("Columns in dataset:")
```

```
Columns in dataset:
```

In [38]:
```
df.columns.tolist()
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 919 entries, 0 to 918
Data columns (total 8 columns):
 #   Column                          Non-Null Count  Dtype 
---  ------                          --------------  ----- 
 0   RespondentID                    919 non-null    int64 
 1   Check_Weather_Daily             919 non-null    object
 2   Check_Weather_Method            919 non-null    object
 3   Weather_Smartwatch_Likelihood   919 non-null    object
 4   Age                             919 non-null    object
 5   Gender                          919 non-null    object
 6   Household_Income                919 non-null    object
 7   US Region                       919 non-null    object
dtypes: int64(1), object(7)
memory usage: 57.6+ KB
```

In [39]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 919 entries, 0 to 918
Data columns (total 8 columns):
 #   Column                          Non-Null Count  Dtype 
---  ------                          --------------  ----- 
 0   RespondentID                    919 non-null    int64 
 1   Check_Weather_Daily             919 non-null    object
 2   Check_Weather_Method            919 non-null    object
 3   Weather_Smartwatch_Likelihood   919 non-null    object
 4   Age                             919 non-null    object
 5   Gender                          919 non-null    object
 6   Household_Income                919 non-null    object
 7   US Region                       919 non-null    object
dtypes: int64(1), object(7)
memory usage: 57.6+ KB
```

In [40]:
```
df.describe()
```

Out[40]:

	RespondentID
count	9.190000e+02
mean	3.879084e+09
std	2.445434e+06
min	3.877567e+09
25%	3.877793e+09
50%	3.878059e+09
75%	3.878700e+09
max	3.887201e+09

In [41]:
```
df['Age'] = pd.to_numeric(df['Age'], errors='coerce')
```

In [42]:
```
Q1 = df['Age'].quantile(0.25)
Q3 = df['Age'].quantile(0.75)
```

```
IQR = Q3 - Q1

lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

outliers = df[(df['Age'] < lower) | (df['Age'] > upper)]

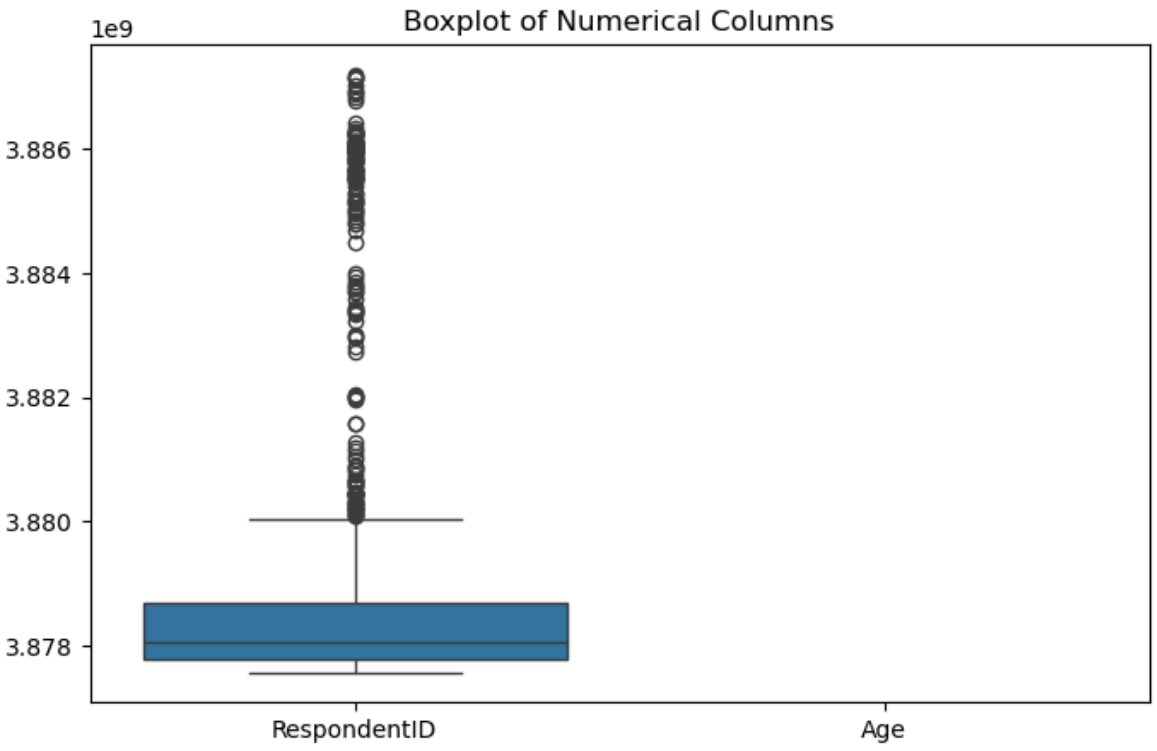
print("Number of outliers:", len(outliers))
print(outliers)
```

```
Number of outliers: 0
Empty DataFrame
Columns: [RespondentID, Check_Weather_Daily, Check_Weather_Method, Weather_Smartw
atch_Likelihood, Age, Gender, Household_Income, US Region]
Index: []
```

In [43]:
```
plt.figure(figsize=(8,5))

sns.boxplot(data=df.select_dtypes(include='number'))

plt.title("Boxplot of Numerical Columns")
plt.show()
```



In [46]:
```
df = pd.read_csv("weather-check.csv")

df.columns = df.columns.str.strip()

df = df.dropna(subset=['Check_Weather_Daily'])

X = df.drop('Check_Weather_Daily', axis=1)
y = df['Check_Weather_Daily']

X = X.drop(['RespondentID', 'Age'], axis=1, errors='ignore')

le = LabelEncoder()
y = le.fit_transform(y)
```

```
categorical_columns = X.select_dtypes(include=['object']).columns
numerical_columns = X.select_dtypes(include=['number']).columns

categorical_transformer = Pipeline([
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('onehot', OneHotEncoder(handle_unknown='ignore'))
])

numerical_transformer = Pipeline([
    ('imputer', SimpleImputer(strategy='median'))
])

preprocessor = ColumnTransformer([
    ('cat', categorical_transformer, categorical_columns),
    ('num', numerical_transformer, numerical_columns)
])

model = Pipeline([
    ('preprocessor', preprocessor),
    ('classifier', LogisticRegression(max_iter=1000))
])

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))

print("\nClassification Report:")
print(classification_report(y_test, y_pred))

cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(6, 4))

sns.heatmap(
    cm,
    annot=True,
    fmt='d',
    cmap='Blues',
    xticklabels=le.classes_,
    yticklabels=le.classes_
)

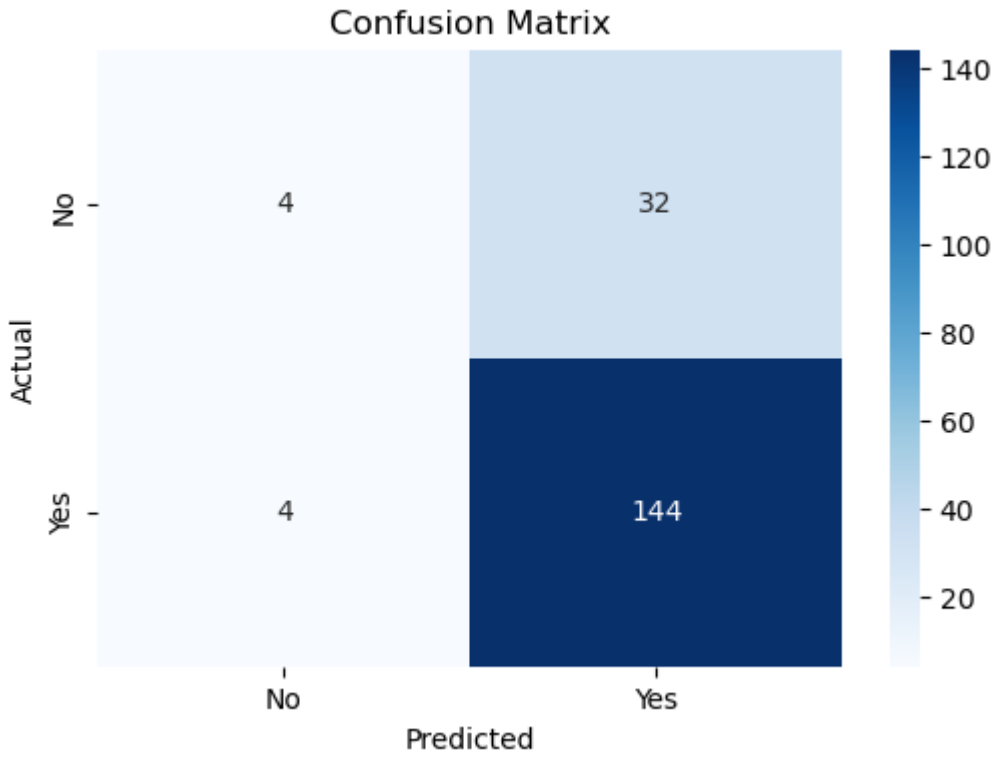
plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```

```
Accuracy: 0.8043478260869565

Classification Report:
              precision    recall  f1-score   support

           0       0.50      0.11      0.18        36
           1       0.82      0.97      0.89       148

    accuracy                           0.80       184
   macro avg       0.66      0.54      0.54       184
weighted avg       0.76      0.80      0.75       184
```



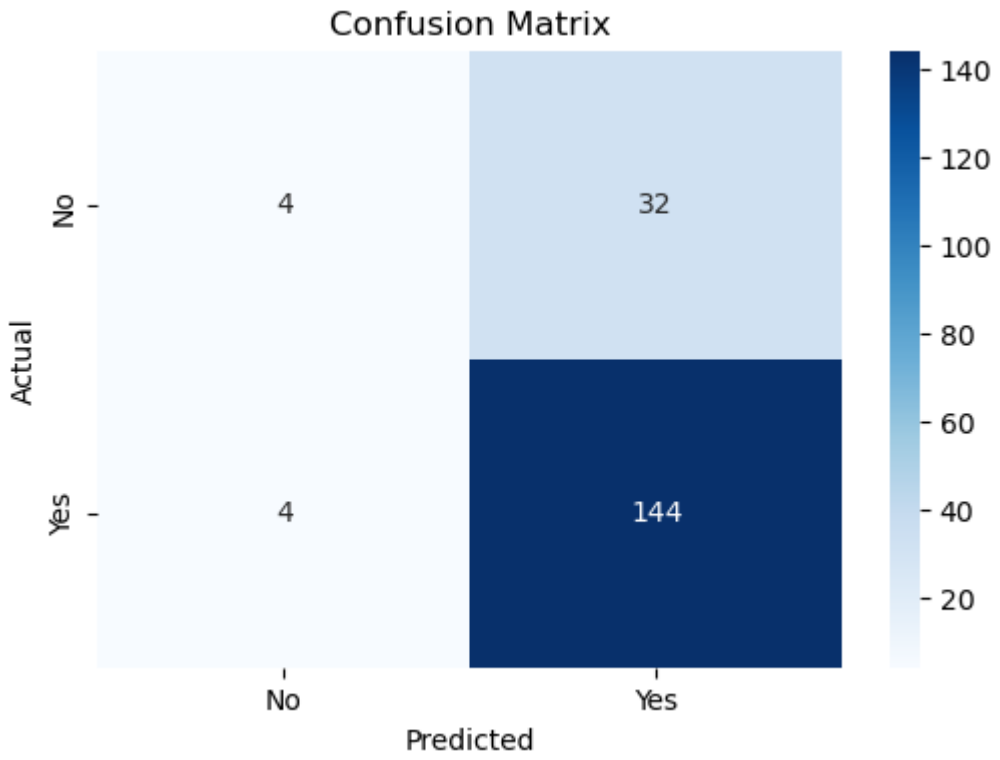
In [47]:

```python
cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(6,4))

sns.heatmap(
    cm,
    annot=True,
    fmt='d',
    cmap='Blues',
    xticklabels=le.classes_,
    yticklabels=le.classes_
)

plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```


Confusion Matrix
No
Yes
Actual
4
32
4
144
No
Yes
Predicted
140
120
100
80
60
40
20

In [ ]: